

I want to build a track and field sprint/jump training-design program as a full-page Google Apps Script web app that opens from its own web link. Later, I want to adapt the same interface so it can also open as a large pop-up window inside a Google Sheet.

The program should be modeled after the older Training Design software screenshots I am uploading. It should be designed specifically for sprint, hurdle, and jump training rather than as a general workout planner.

Show more ▾

Thought for 5m 4s >

Downloadable copies are included as a backup:

[Download Code.gs](#)

[Download Index.html](#)

Code.gs

Replace everything currently inside `Code.gs` with this:

JavaScript



```
const APP_VERSION = '1.0.0';
const STORAGE_PROPERTY = 'TRACK_TRAINING_DESIGN_SPREADSHEET_ID';
const STORAGE_FILE_NAME = 'Track and Field Training Design - Data';
const PLANS_SHEET_NAME = 'Plans';
const DAYS_SHEET_NAME = 'Days';

const PLAN_HEADERS = [
  'Plan ID',
  'Plan Name',
  'Season Type',
  'Custom Season Type',
  'Training Group',
  'Custom Group Name',
  'Start Date',
  'End Date',
  'Mesocycles JSON',
  'Created At',
  'Updated At'
];

const DAY_HEADERS = [
  'Plan ID',
```



```

    return plan;
}

function savePlan(planInput) {
    const lock = LockService.getScriptLock();
    lock.waitLock(30000);

    try {
        const spreadsheet = getStorageSpreadsheet_();
        const plan = normalizePlan_(planInput);
        const existing = plan.id ? readPlan_(spreadsheet, plan.id) : null;
        const now = new Date().toISOString();

        if (!plan.id) {
            plan.id = Utilities.getUuid();
        }

        plan.createdAt = existing ? existing.createdAt : now;
        plan.updatedAt = now;

        upsertPlan_(spreadsheet, plan);
        replacePlanDays_(spreadsheet, plan);
        SpreadsheetApp.flush();

        return {
            success: true,
            plan: summarizePlan_(plan),
            message: 'Plan saved.'
        };
    } finally {
        lock.releaseLock();
    }
}

function duplicatePlan(planId) {
    const lock = LockService.getScriptLock();
    lock.waitLock(30000);

    try {
        const spreadsheet = getStorageSpreadsheet_();
        const original = readPlan_(spreadsheet, String(planId || ''));
        if (!original) {
            throw new Error('That plan could not be found.');
```

```

    upsertPlan_(spreadsheet, copy);
    replacePlanDays_(spreadsheet, copy);
    SpreadsheetApp.flush();

    return {
      success: true,
      plan: summarizePlan_(copy),
      message: 'Plan duplicated.'
    };
  } finally {
    lock.releaseLock();
  }
}

function deletePlan(planId) {
  const lock = LockService.getScriptLock();
  lock.waitLock(30000);

  try {
    const spreadsheet = getStorageSpreadsheet_();
    const id = String(planId || '');
    if (!id) {
      throw new Error('A plan ID is required.');
```

```

    } catch (error) {
      properties.deleteProperty(STORAGE_PROPERTY);
    }
  }

const spreadsheet = SpreadsheetApp.create(STORAGE_FILE_NAME);
properties.setProperty(STORAGE_PROPERTY, spreadsheet.getId());
ensureSchema_(spreadsheet);

try {
  DriveApp.getFileById(spreadsheet.getId()).setDescription(
    'Automatically created storage for the Track & Field Training Design web app.'
  );
} catch (error) {
  // The app still works if Drive description permissions are restricted.
}

return spreadsheet;
}

function ensureSchema_(spreadsheet) {
  let plansSheet = spreadsheet.getSheetByName(PLANS_SHEET_NAME);
  const sheets = spreadsheet.getSheets();

  if (!plansSheet) {
    if (sheets.length === 1 && sheets[0].getLastRow() === 0) {
      plansSheet = sheets[0];
      plansSheet.setName(PLANS_SHEET_NAME);
    } else {
      plansSheet = spreadsheet.insertSheet(PLANS_SHEET_NAME);
    }
  }

  let daysSheet = spreadsheet.getSheetByName(DAYS_SHEET_NAME);
  if (!daysSheet) {
    daysSheet = spreadsheet.insertSheet(DAYS_SHEET_NAME);
  }

  formatStorageSheet_(plansSheet, PLAN_HEADERS);
  formatStorageSheet_(daysSheet, DAY_HEADERS);

  if (!daysSheet.isSheetHidden()) {
    daysSheet.hideSheet();
  }
}

function formatStorageSheet_(sheet, headers) {
  const currentHeaders = sheet.getRange(1, 1, 1, headers.length).getValues()[0];
  const needsHeaders = headers.some((header, index) => currentHeaders[index] !== header);

  if (needsHeaders) {
    sheet.getRange(1, 1, 1, headers.length).setValues([headers]);
  }
}

```

```

}

sheet.setFrozenRows(1);
sheet.getRange(1, 1, 1, headers.length)
    .setFontWeight('bold')
    .setBackground('#0f172a')
    .setFontColor('ffffff');
sheet.autoResizeColumns(1, headers.length);
}

function listPlans_(spreadsheet) {
    const plansSheet = spreadsheet.getSheetByName(PLANS_SHEET_NAME);
    const daysSheet = spreadsheet.getSheetByName(DAYS_SHEET_NAME);
    const planRows = getRowsBelowHeader_(plansSheet, PLAN_HEADERS.length);
    const dayRows = getRowsBelowHeader_(daysSheet, DAY_HEADERS.length);
    const statsByPlan = {};

    dayRows.forEach(row => {
        const planId = String(row[0] || '');
        if (!statsByPlan[planId]) {
            statsByPlan[planId] = { classifiedDays: 0, competitions: 0 };
        }
        if (String(row[2] || 'None') !== 'None') {
            statsByPlan[planId].classifiedDays += 1;
        }
        if (toBoolean_(row[3])) {
            statsByPlan[planId].competitions += 1;
        }
    });

    return planRows
        .filter(row => row[0])
        .map(row => {
            const plan = planFromRow_(row);
            const stats = statsByPlan[plan.id] || { classifiedDays: 0, competitions: 0 };
            return Object.assign(summarizePlan_(plan), stats);
        })
        .sort((a, b) => String(b.updatedAt).localeCompare(String(a.updatedAt)));
}

function readPlan_(spreadsheet, planId) {
    if (!planId) return null;

    const plansSheet = spreadsheet.getSheetByName(PLANS_SHEET_NAME);
    const planRows = getRowsBelowHeader_(plansSheet, PLAN_HEADERS.length);
    const row = planRows.find(item => String(item[0]) === String(planId));
    if (!row) return null;

    const plan = planFromRow_(row);
    const daysSheet = spreadsheet.getSheetByName(DAYS_SHEET_NAME);
    const dayRows = getRowsBelowHeader_(daysSheet, DAY_HEADERS.length);
    plan.days = {};

```

```

dayRows.forEach(dayRow => {
  if (String(dayRow[0]) !== String(planId)) return;
  const date = String(dayRow[1] || '');
  if (!date) return;

  plan.days[date] = {
    workload: WORKLOADS.includes(String(dayRow[2])) ? String(dayRow[2]) : 'None',
    competition: toBoolean_(dayRow[3]),
    competitionName: String(dayRow[4] || ''),
    notes: String(dayRow[5] || '')
  };
});

return plan;
}

function planFromRow_(row) {
  let mesocycles = [];
  try {
    mesocycles = JSON.parse(String(row[8] || '[]'));
    if (!Array.isArray(mesocycles)) mesocycles = [];
  } catch (error) {
    mesocycles = [];
  }

  return {
    id: String(row[0] || ''),
    planName: String(row[1] || ''),
    seasonType: String(row[2] || ''),
    customSeasonType: String(row[3] || ''),
    trainingGroup: String(row[4] || ''),
    customGroupName: String(row[5] || ''),
    startDate: String(row[6] || ''),
    endDate: String(row[7] || ''),
    mesocycles: mesocycles,
    createdAt: String(row[9] || ''),
    updatedAt: String(row[10] || ''),
    days: {}
  };
}

function upsertPlan_(spreadsheet, plan) {
  const sheet = spreadsheet.getSheetByName(PLANS_SHEET_NAME);
  const rows = getRowsBelowHeader_(sheet, PLAN_HEADERS.length);
  const rowIndex = rows.findIndex(row => String(row[0]) === String(plan.id));
  const values = [
    plan.id,
    plan.planName,
    plan.seasonType,
    plan.customSeasonType,
    plan.trainingGroup,

```

```

    plan.customGroupName,
    plan.startDate,
    plan.endDate,
    JSON.stringify(plan.mesocycles || []),
    plan.createdAt,
    plan.updatedAt
  ]];

  if (rowIndex === -1) {
    sheet.getRange(sheet.getLastRow() + 1, 1, 1, PLAN_HEADERS.length).setValues(values);
  } else {
    sheet.getRange(rowIndex + 2, 1, 1, PLAN_HEADERS.length).setValues(values);
  }
}

function replacePlanDays_(spreadsheet, plan) {
  const daysSheet = spreadsheet.getSheetByName(DAYS_SHEET_NAME);
  const existingRows = getRowsBelowHeader_(daysSheet, DAY_HEADERS.length)
    .filter(row => String(row[0]) !== String(plan.id));

  const newRows = Object.keys(plan.days || {}).
    .sort()
    .map(date => {
      const day = plan.days[date];
      return [
        plan.id,
        date,
        day.workload,
        Boolean(day.competition),
        day.competitionName,
        day.notes
      ];
    })
    .filter(row => row[2] !== 'None' || row[3] || row[4] || row[5]);

  rewriteDayRows_(daysSheet, existingRows.concat(newRows));
}

function removePlanDays_(spreadsheet, planId) {
  const daysSheet = spreadsheet.getSheetByName(DAYS_SHEET_NAME);
  const remainingRows = getRowsBelowHeader_(daysSheet, DAY_HEADERS.length)
    .filter(row => String(row[0]) !== String(planId));
  rewriteDayRows_(daysSheet, remainingRows);
}

function rewriteDayRows_(sheet, rows) {
  const existingRowCount = Math.max(sheet.getLastRow() - 1, 0);
  if (existingRowCount > 0) {
    sheet.getRange(2, 1, existingRowCount, DAY_HEADERS.length).clearContent();
  }
  if (rows.length > 0) {
    sheet.getRange(2, 1, rows.length, DAY_HEADERS.length).setValues(rows);
  }
}

```



```

    }
  }

function summarizePlan_(plan) {
  return {
    id: plan.id,
    planName: plan.planName,
    seasonType: plan.seasonType,
    customSeasonType: plan.customSeasonType,
    trainingGroup: plan.trainingGroup,
    customGroupName: plan.customGroupName,
    startDate: plan.startDate,
    endDate: plan.endDate,
    createdAt: plan.createdAt,
    updatedAt: plan.updatedAt
  };
}

function normalizePlan_(input) {
  const source = input || {};
  const planName = cleanText_(source.planName, 120).trim();
  const seasonType = cleanText_(source.seasonType, 50);
  const customSeasonType = cleanText_(source.customSeasonType, 80);
  const trainingGroup = cleanText_(source.trainingGroup, 100);
  const customGroupName = cleanText_(source.customGroupName, 100);
  const startDate = normalizeDate_(source.startDate);
  const endDate = normalizeDate_(source.endDate);

  if (!planName) throw new Error('Enter a plan name before saving.');
```

```

  if (!SEASON_TYPES.includes(seasonType)) throw new Error('Select a valid season type.');
```

```

  if (seasonType === 'Custom' && !customSeasonType.trim()) {
    throw new Error('Enter a name for the custom season type.');
```

```

  }

  if (!TRAINING_GROUPS.includes(trainingGroup)) throw new Error('Select a valid training
  if (trainingGroup === 'Custom Group' && !customGroupName.trim()) {
    throw new Error('Enter a name for the custom training group.');
```

```

  }

  if (dateDiffInDays_(startDate, endDate) < 0) {
    throw new Error('The end date must be on or after the start date.');
```

```

  }

  if (dateDiffInDays_(startDate, endDate) > 550) {
    throw new Error('Version 1 supports plans of up to 550 days.');
```

```

  }

  const mesocycles = Array.isArray(source.mesocycles)
    ? source.mesocycles.slice(0, 100).map((item, index) => {
        const mesoStart = normalizeDate_(item.startDate || startDate);
        const mesoEnd = normalizeDate_(item.endDate || endDate);
        if (dateDiffInDays_(mesoStart, mesoEnd) < 0) {
          throw new Error('Mesocycle ' + (index + 1) + ' has an end date before its start
        }
      })
    : []
  return {

```

```

        id: cleanText_(item.id || Utilities.getUuid(), 80),
        name: cleanText_(item.name || ('Mesocycle ' + (index + 1)), 80),
        startDate: mesoStart,
        endDate: mesoEnd
    });
    })
    : [];

const days = {};
const inputDays = source.days && typeof source.days === 'object' ? source.days : {};

Object.keys(inputDays).forEach(dateKey => {
    let normalizedDate;
    try {
        normalizedDate = normalizeDate_(dateKey);
    } catch (error) {
        return;
    }

    if (!dateIsWithinRange_(normalizedDate, startDate, endDate)) return;

    const day = inputDays[dateKey] || {};
    const workload = WORKLOADS.includes(String(day.workload)) ? String(day.workload) : 'None';
    const competition = Boolean(day.competition);
    const competitionName = cleanText_(day.competitionName, 160);
    const notes = cleanText_(day.notes, 5000);

    if (workload !== 'None' || competition || competitionName || notes) {
        days[normalizedDate] = {
            workload: workload,
            competition: competition,
            competitionName: competitionName,
            notes: notes
        };
    }
});

return {
    id: cleanText_(source.id, 80),
    planName: planName,
    seasonType: seasonType,
    customSeasonType: customSeasonType,
    trainingGroup: trainingGroup,
    customGroupName: customGroupName,
    startDate: startDate,
    endDate: endDate,
    mesocycles: mesocycles,
    days: days,
    createdAt: cleanText_(source.createdAt, 40),
    updatedAt: cleanText_(source.updatedAt, 40)
};
}

```

```

function getRowsBelowHeader_(sheet, columnCount) {
  const rowCount = Math.max(sheet.getLastRow() - 1, 0);
  if (rowCount === 0) return [];
  return sheet.getRange(2, 1, rowCount, columnCount).getValues();
}

function normalizeDate_(value) {
  const text = String(value || '');
  const match = /^(\d{4})-(\d{2})-(\d{2})$/.exec(text);
  if (!match) throw new Error('Use valid start and end dates.');
```

```

  const year = Number(match[1]);
  const month = Number(match[2]);
  const day = Number(match[3]);
  const date = new Date(Date.UTC(year, month - 1, day));

  if (
    date.getUTCFullYear() !== year ||
    date.getUTCMonth() !== month - 1 ||
    date.getUTCDate() !== day
  ) {
    throw new Error('Use valid start and end dates.');
```

```

  }

  return text;
}

function dateDiffInDays_(startDate, endDate) {
  const start = new Date(startDate + 'T00:00:00Z');
  const end = new Date(endDate + 'T00:00:00Z');
  return Math.round((end.getTime() - start.getTime()) / 86400000);
}

function dateIsWithinRange_(date, startDate, endDate) {
  return date >= startDate && date <= endDate;
}

function cleanText_(value, maximumLength) {
  return String(value == null ? '' : value).slice(0, maximumLength);
}

function toBoolean_(value) {
  return value === true || String(value).toLowerCase() === 'true';
}

```

Index.html

Click the **+** beside **Files**, choose **HTML**, and name the file exactly:



Replace everything inside the new `Index.html` file with this:

HTML



```
<!DOCTYPE html>
<html>
  <head>
    <base target="_top">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Track & Field Training Design</title>
    <style>
      :root {
        --navy: #0f172a;
        --navy-soft: #1e293b;
        --slate: #475569;
        --muted: #64748b;
        --line: #dbe3ee;
        --panel: #ffffff;
        --page: #f4f7fb;
        --blue: #2563eb;
        --blue-dark: #1d4ed8;
```

Setup and deployment

1. Go to script.google.com and create a **New project**.
2. Replace the existing contents of `Code.gs` with the first code block.
3. Click the **+** beside **Files** and choose **HTML**.
4. Name the file:



5. Replace the contents of `Index.html` with the second code block.
6. Click **Save project**.
7. Click **Deploy** in the upper-right corner.
8. Choose **New deployment**.
9. Click the gear icon beside **Select type** and choose **Web app**.
10. Use these settings:

```
Description: Training Design Version 1
Execute as: Me
Who has access: Only myself
```



11. Click **Deploy**.

12. Complete the Google authorization prompts.

13. Copy the web-app URL and open it in a new browser tab.

The first time the web app opens, it automatically creates a Google Sheet in your Drive.

Index.html

```
<!DOCTYPE html>
<html>
  <head>
    <base target="_top">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Track & Field Training Design</title>
    <style>
      :root {
        --navy: #0f172a;
        --navy-soft: #1e293b;
        --slate: #475569;
        --muted: #64748b;
        --line: #dbe3ee;
        --panel: #ffffff;
        --page: #f4f7fb;
        --blue: #2563eb;
        --blue-dark: #1d4ed8;
```

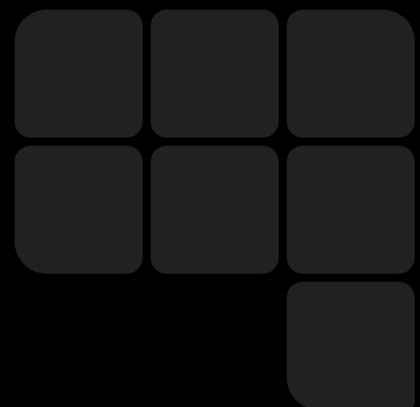
replace? I don't even have anything yet? you just started

I set the start and end dates, the weekly calendar appears,
I put intensities for each day, click save, go back to edit,
nothing saves



Want to make a new one using this one as a base but adding features like you see in these images.

when I click on the blue button that says microcycles it just does this. nothing happens:



Use these to help inform the design of the application

Pensó durante 6s >

Good. I think we have enough reference material now.

The next smart move is **not more coding yet**. The next move should be to lock the design for **Version 3** so the app doesn't become messy.

I'd build Version 3 around this structure:

Version 3: Weekly Practice Builder

Core screens:

1. Yearly Plan

Keep what already works.

2. Microcycle Planner

Keep the demand numbers, colors, graph, and themes.

3. Weekly Practice Builder

This is the new main screen:

- week number
- week type
- mesocycle theme
- microcycle theme
- daily demand number
- equipment
- warmup
- main work
- event-group blocks
- cooldown
- coach notes
- athlete notes

4. Template Library

- indoor templates
- outdoor templates
- AC400-style templates
- routine / warmup library

5. Print Views

- coach view
- athlete view

The key is this: the app should let you **load a template**, then **customize the actual week**, instead of making you type everything from scratch.

Next I'd make a clean Version 3 blueprint with the exact fields, buttons, screens, and storage tabs before writing the next code.

Ok

Pensó durante 6s >

Absolutely. Here is the **Version 3 blueprint** before we touch the code.

Version 3: Weekly Practice Builder

Main purpose

Version 3 should turn the app from a yearly/microcycle planning tool into a real **weekly practice-building system**.

The workflow should be:

```
Choose yearly plan
→ choose week
→ set demand pattern
→ load template or start blank
→ build each day
→ print coach view or athlete view
```



1. Main screens

Screen 1: Home / Saved Plans

Keep mostly the same.

Each saved plan should have buttons for:

```
Edit Year Plan
Microcycles
Weekly Practice Builder
Duplicate
Delete
```



Screen 2: Yearly Plan

Keep the current yearly calendar.

This remains the big-picture planning layer:

Mesocycles
Competition dates
Daily workload classifications
Season start/end
Training group



No major changes needed here yet.

Screen 3: Microcycle Planner

Keep the Version 2 idea:

Week selector
Mesocycle theme
Microcycle theme
Daily demand scores 0-10
Color-coded demand number boxes
Weekly demand graph
Weekly total
Weekly average
Major / Maintain / Minor objectives



The demand number should stay central.

Example:

Mon	Tue	Wed	Thu	Fri	Sat	Sun
7	4	6	5	3	9	0



Screen 4: Weekly Practice Builder

This becomes the main new Version 3 screen.

Each week should have a header:

Plan Name
Season Type
Week Number
Week Date Range
Week Type
Mesocycle Theme
Microcycle Theme
Category
Training Group



Week Type options

Normal Training Week
Routine Building Week
Recovery Week
Meet Week
Championship Week
Travel Week
Winter Break / Holiday Week
Testing Week
Custom



This comes directly from how your indoor and outdoor documents are structured. You have normal training weeks, routine-building weeks, meet weeks, and altered weeks around holidays or championship meets. 

Google Keep Document

 Copy of Outdoor 2026 Weekly Tra...

2. Daily Practice Builder structure

Each day should not be one big text box.

Each day should have structured sections:

Day Theme
Demand Score
Equipment
Setup / Organization
Warmup
Main Work
Event Group Blocks
Relays
Jumps
Hurdles
Weights / Strength
Cooldown
Coach Notes
Athlete Notes
Competition Schedule



Example day structure

Monday
Theme: Acceleration
Demand: 7



Equipment: med balls, hurdles, blocks

Setup / Organization

1. Direction of intention
2. Agenda
3. Bathroom
4. 2 lap jog
5. Stretch

Warmup

Acceleration Warm Up

Main Work

3 x SLJ

5 x OHB

Block 15, 25, 35

Block 20, 30, 40

Hurdles

3 x H1, H2, H3

Relays

4x2, 4x4 handoffs

Cooldown

Hurdle Mobility

That matches the way your Outdoor 2026 and Indoor 2025–2026 plans are actually written.



Copy of Outdoor 2026 Weekly Tra...



Google Keep Document

3. Event-group blocks

Each daily plan should allow group-specific sections.

Default group blocks:

Everyone

Short Sprints

Long Sprints

Sprint Hurdles

Long Hurdles / 300H

Long Jump / Triple Jump

High Jump

Relays

Custom Group



This is important because your real plans often split the same practice into SS, LS, SH, jumpers, hurdlers, and relay work. 

Google Keep Document

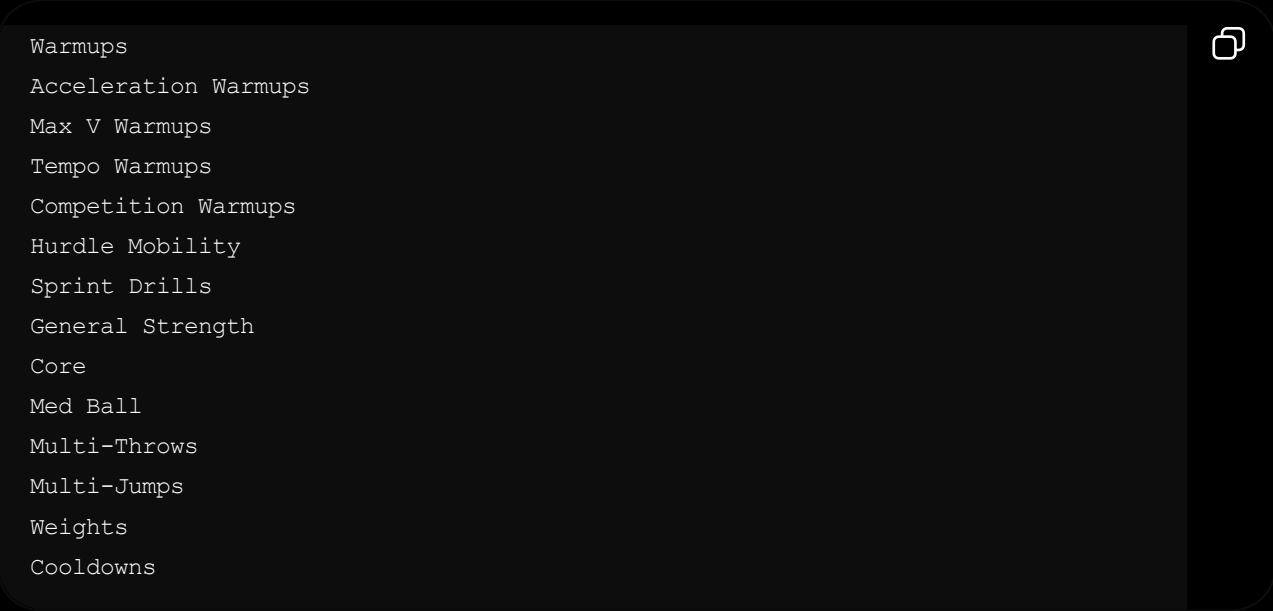
4. Template system

Version 3 should have two template libraries.

A. Routine Library

This comes from your routines/warmups document.

Categories:



Warmups
Acceleration Warmups
Max V Warmups
Tempo Warmups
Competition Warmups
Hurdle Mobility
Sprint Drills
General Strength
Core
Med Ball
Multi-Throws
Multi-Jumps
Weights
Cooldowns



The routines document already contains warmups, sprint development drills, hurdle mobility, general strength circuits, med ball circuits, multi-jump circuits, and weight-room categories. 

Copy of Routines and warm ups 2...

 Copy of Routines and warm ups 2...

 Copy of Routines and warm ups 2...

Buttons needed



Insert Routine
Preview Routine
Edit After Insert
Save as Custom Routine



B. Weekly Template Library

This comes from the AC400-MM and your implementation documents.

Template groups:

AC400-MM Progression
Indoor 2025-2026
Outdoor 2026
Custom Saved Templates



Template actions:

Load Full Week
Load One Day
Save Current Week as Template
Duplicate Template
Delete Custom Template



The AC400-MM document is organized by mesocycles and weekly day-by-day progressions, so it is ideal for “Load Week Template.”



Copy of AC400-MM-Workouts

5. Print views

Version 3 should include two printable views.

Coach View

Includes everything:

Equipment
Setup reminders
Bathroom / organization notes
Shin checks
Athlete-specific reminders
Relay notes
Event group details
Coach notes
Competition schedule



Athlete View

Cleaner version:

Day
Theme
Warmup



Main work
Group assignments
Cooldown
Meet schedule if needed

This matters because your real working documents include coaching logistics that you may not always want printed for athletes.

6. Storage tabs in the hidden Google Sheet

Version 3 should add new tabs behind the scenes:

Plans
Days
Weeks
PracticeDays
PracticeBlocks
RoutineLibrary
WeekTemplates



What each tab stores

Plans
Saved yearly plans

Days
Yearly plan calendar classifications

Weeks
Microcycle themes, demand numbers, objectives

PracticeDays
Daily theme, equipment, notes, demand score

PracticeBlocks
The actual workout sections for each day

RoutineLibrary
Reusable warmups, circuits, drills, cooldowns

WeekTemplates
Saved full-week templates



7. Version 3 build order

To avoid breaking what already works, build it in this order:

Version 3A — Weekly Practice Builder

Add:

```
Weekly Practice Builder button
Week header
Monday-Sunday editor
Structured day sections
Save/load practice week
Coach notes
Athlete notes
```



No big routine library yet.

Version 3B — Routine Library

Add:

```
Routine library panel
Insert routine into selected day
Built-in warmup/core/mobility examples
Save custom routine
```



Version 3C — Weekly Templates

Add:

```
Load week template
Save current week as template
Indoor / Outdoor / AC400 template categories
```



Version 3D — Print Views

Add:

```
Coach View
Athlete View
Print button
Clean weekly layout
```



My recommendation

Build Version 3A first.

That means the next code update should add the **Weekly Practice Builder** without trying to fully import every routine and template yet.

Once that structure works, adding the routine library and template library will be much cleaner.

Ok

Pensó durante 4m 56s >

Built Version 3A.

Use these files:

[Download Version 3A ZIP](#)

Plain-text copies:

[Code.gs Version 3A](#)

[Index.html Version 3A](#)

What Version 3A adds

It keeps your current yearly plan and microcycle system, and adds a new **Weekly Practice Builder** with:

- Practice Builder button on each plan
- Week selector
- Week type
- Category
- Training group label
- Mesocycle theme
- Microcycle theme
- Monday–Sunday practice tabs
- Daily demand score
- Equipment
- Setup / organization
- Warmup
- Main work
- Event group blocks
- Relays
- Jumps
- Hurdles

- Weights / strength
- Cooldown
- Coach notes
- Athlete notes
- Competition schedule
- Coach preview / print view
- Standard daily skeleton button

It also adds two new hidden storage tabs behind the scenes:

```
PracticeWeeks
PracticeDays
```



Update steps

1. Open your Apps Script project.
2. Open `Code.gs`.
3. Press **Ctrl+A**.
4. Delete everything.
5. Paste the full contents from **Code.gs Version 3A**.
6. Open `Index.html`.
7. Press **Ctrl+A**.
8. Delete everything.
9. Paste the full contents from **Index.html Version 3A code**.
10. Click **Save**.
11. Click **Deploy**.
12. Choose **Manage deployments**.
13. Click the pencil icon.
14. Under **Version**, choose **New version**.
15. Click **Deploy**.
16. Refresh your existing web-app link.

Your existing yearly plans and microcycles should remain.

looks good:

everything works

Thought for a few seconds >

Great. That means Version 3A is stable:

- yearly plans work
- microcycles work
- practice builder opens
- practice weeks save and reload
- coach preview works
- print view works

Next logical step is **Version 3B: Routine Library**.

That would add a panel where you can insert prebuilt routines into the selected day, such as:

```
Acceleration Warm Up
Max V Warm Up
30m Warm Up
Sallies Warm Up
Competition Warm Up
Hurdle Mobility
Hamstring Cool Down
PVC Cool Down
Core X
Gemini
Monster
General Strength Circuit 1
General Strength Circuit 2
```



The goal would be: instead of typing “Acceleration Warm Up” or “Hurdle Mobility” every time, you click the routine and choose where to insert it: **Warmup, Main Work, Cooldown, Everyone, Short Sprints, Long Sprints, Hurdles, Jumps**, etc.

That is the next piece that will make this start feeling like a real training-design program instead of just a structured notes app.

ok

Thought for 3m 19s >

Built Version 3B.

Use these files:

Download Version 3B ZIP

Plain-text copies:

Code.gs Version 3B

Index.html Version 3B

What Version 3B adds

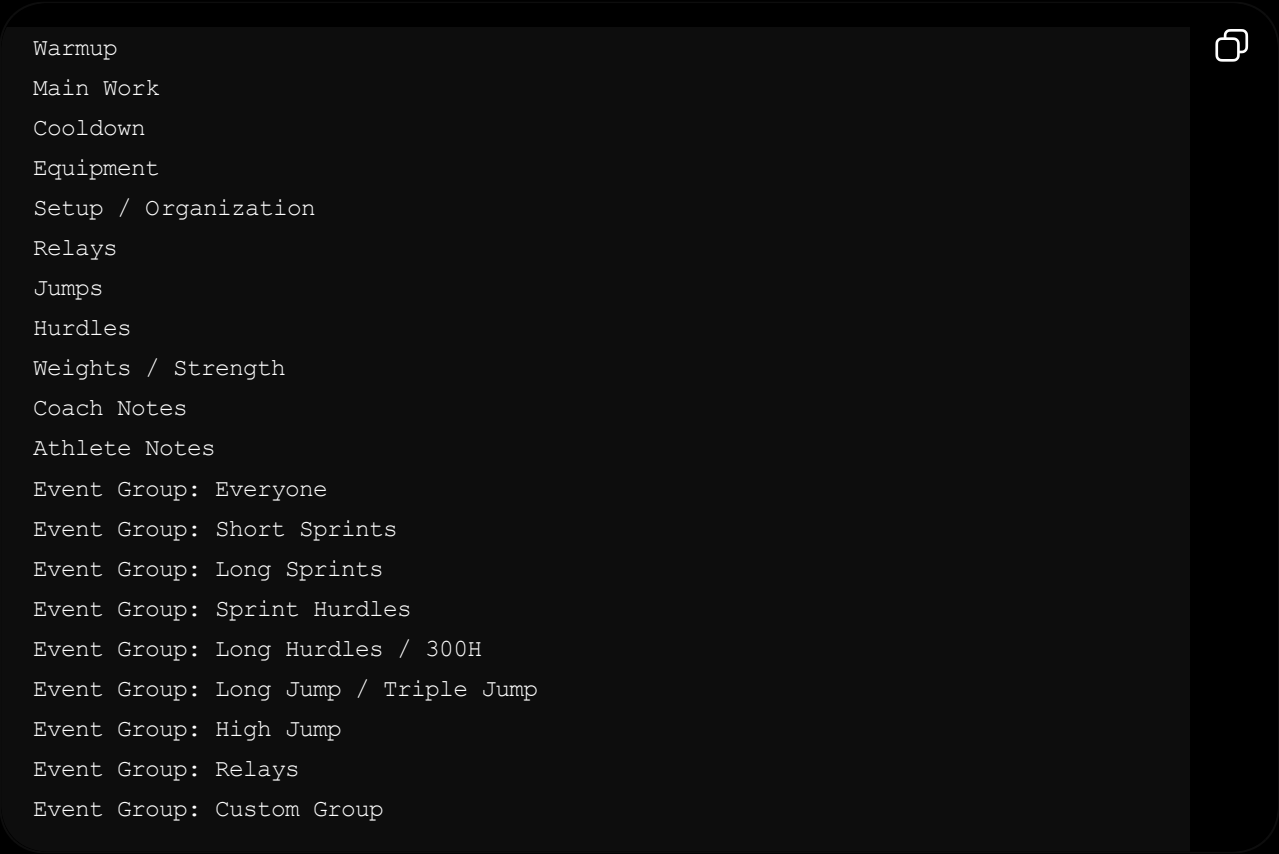
This keeps everything from Version 3A and adds a **Routine Library** inside the Practice Builder.

You can now insert prebuilt routines into the selected day:

- 30m Warm Up
- Acceleration Warm Up
- Max V Warm Up
- Sallies Warm Up
- Competition Warm Up
- Tempo Warm Up
- Hurdle Mobility Cool Down
- Hamstring Cool Down
- PVC Cool Down
- Cancer Core
- Core X

- Monster
- General Strength Circuit 1
- General Strength Circuit 2
- Killer Pillar
- Med Ball Throws Basic
- Basic Jump Power Menu
- Jumpers Approach Work
- Hurdle Technical Block
- Relay Handoff Block
- Tempo Options
- Acceleration Options
- Max Velocity Options

You can choose where the routine inserts:



Warmup
Main Work
Cooldown
Equipment
Setup / Organization
Relays
Jumps
Hurdles
Weights / Strength
Coach Notes
Athlete Notes
Event Group: Everyone
Event Group: Short Sprints
Event Group: Long Sprints
Event Group: Sprint Hurdles
Event Group: Long Hurdles / 300H
Event Group: Long Jump / Triple Jump
Event Group: High Jump
Event Group: Relays
Event Group: Custom Group

Update steps

1. Open your Apps Script project.
2. Replace all of `Code.gs` with **Code.gs Version 3B**.
3. Replace all of `Index.html` with **Index.html Version 3B**.
4. Click **Save**.
5. Go to **Deploy** → **Manage deployments**.
6. Click the pencil icon.
7. Under **Version**, choose **New version**.

8. Click **Deploy**.

9. Refresh your existing web-app link.

Your saved plans, microcycles, and practice weeks should remain.



I want to be able to click on drop downs that let me choose the different aspects of each day. Also, I don't know why the acceleration warm up was posted above the warm up. I guess I'm a little confused about the layout and functionality

Preparing app tools ▾

Thought for 5s >

Not directly, at least not through the tools I have here.

I can edit **Google Docs, Sheets, and Slides** directly, but I don't have a tool that can open and modify the actual **Google Apps Script project files** like `Code.gs` and `Index.html`

inside the Apps Script editor.

So for now, the update process still has to be:

1. I give you the corrected `Code.gs` and `Index.html`.
2. You paste them into Apps Script.
3. You redeploy a new version.

What I *can* do to make it easier is stop giving download links first and instead give you exactly:

Copy this entire `Code.gs`



and

Copy this entire `Index.html`



right in the chat, so you do not have to open separate text files unless you want the backup.

no, keep the text files, they work fine

Thought for 4s >

Got it — I'll keep doing it that way:

- ZIP file as backup
- `Code.gs` as a plain-text file
- `Index.html` as a plain-text file

For the next update, I'll make **Version 3B.1** with the dropdown-builder fix and cleaner routine insertion.

ok

Thought for 2m 38s >

Built **Version 3B.1 — Dropdown Builder Fix.**

Use these files:

Download Version 3B.1 ZIP

Plain-text copies:

Code.gs Version 3B.1

Index.html Version 3B.1

What this fixes/adds

- **Acceleration Warm Up** now inserts into Warmup by default
- Cooldowns insert into **Cooldown**
- Relays insert into **Relays**
- Jumps insert into **Jumps**
- Hurdles insert into **Hurdles**
- Added section-specific dropdowns directly inside each day
- Added a **Quick Daily Builder** area at the top of the day editor
- Added dropdowns for:
 - Theme
 - Equipment
 - Setup
 - Warmup
 - Main Work
 - Cooldown
 - Relays
 - Jumps
 - Hurdles
 - Weights / Strength
- The main Routine Library now defaults to each routine's correct section instead of sending everything wherever the target dropdown happens to be set.

Update steps

1. Replace all of `Code.gs` with **Code.gs Version 3B.1**.
2. Replace all of `Index.html` with **Index.html Version 3B.1**.
3. Save.
4. Go to **Deploy** → **Manage deployments**.
5. Click the pencil icon.
6. Choose **New version**.
7. Click **Deploy**.
8. Refresh the same web-app link.